CHAPTER – 1

Name: Jeslet Jessica. T

Regno: 192524135

Course: CSA0603 – DAA

EXPERIMENT - 1

main.py

```python
def first_palindrome(words):
    for word in words:
        # check if word is same forward and backward
        if word == word[::-1]:
            return word
    return ""  # if no palindrome found

# Example 1
words1 = ["abc","car","ada","racecar","cool"]
print(first_palindrome(words1))   # Output: ada

# Example 2
words2 = ["notapalindrome","racecar"]
print(first_palindrome(words2))   # Output: racecar
```

Output

```
ada
racecar

=== Code Execution Successful ===
```

EXPERIMENT – 2

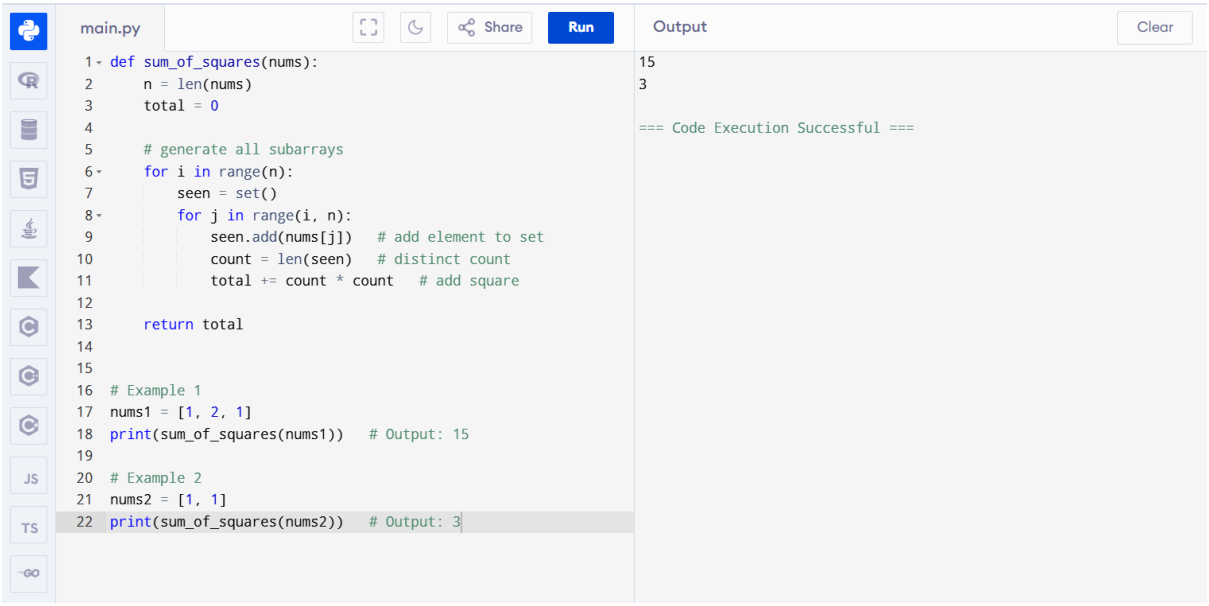
main.py

```python
def find_counts(nums1, nums2):
    answer1 = 0
    answer2 = 0

    # count elements of nums1 present in nums2
    for x in nums1:
        if x in nums2:
            answer1 += 1

    # count elements of nums2 present in nums1
    for x in nums2:
        if x in nums1:
            answer2 += 1

    return [answer1, answer2]


# Example 1
nums1 = [2, 3, 2]
nums2 = [1, 2]
print(find_counts(nums1, nums2))   # Output: [2, 1]

# Example 2
nums1 = [4, 3, 2, 3, 1]
nums2 = [2, 2, 5, 2, 3, 6]
print(find_counts(nums1, nums2))   # Output: [3, 4]
```

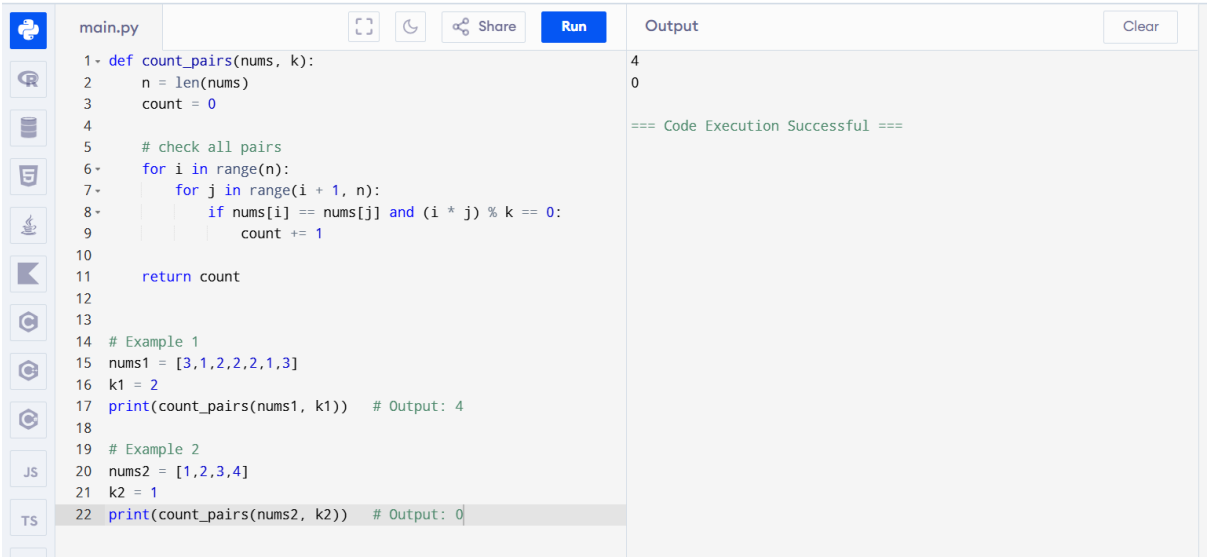
Output

```
[2, 1]
[3, 4]

=== Code Execution Successful ===
```

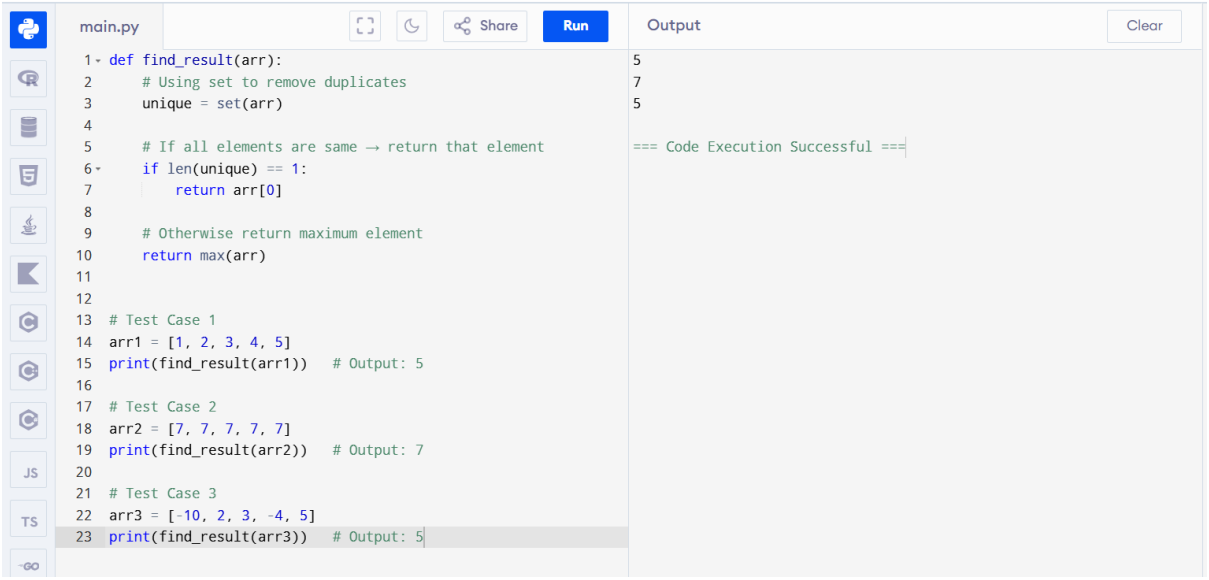
EXPERIMENT – 3



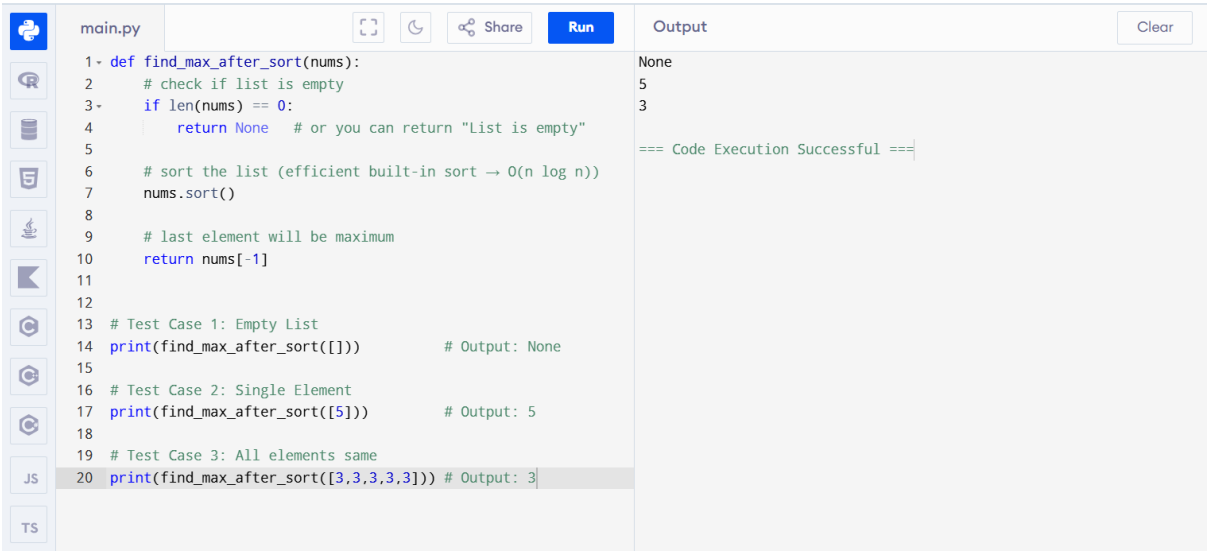
EXPERIMENT – 4



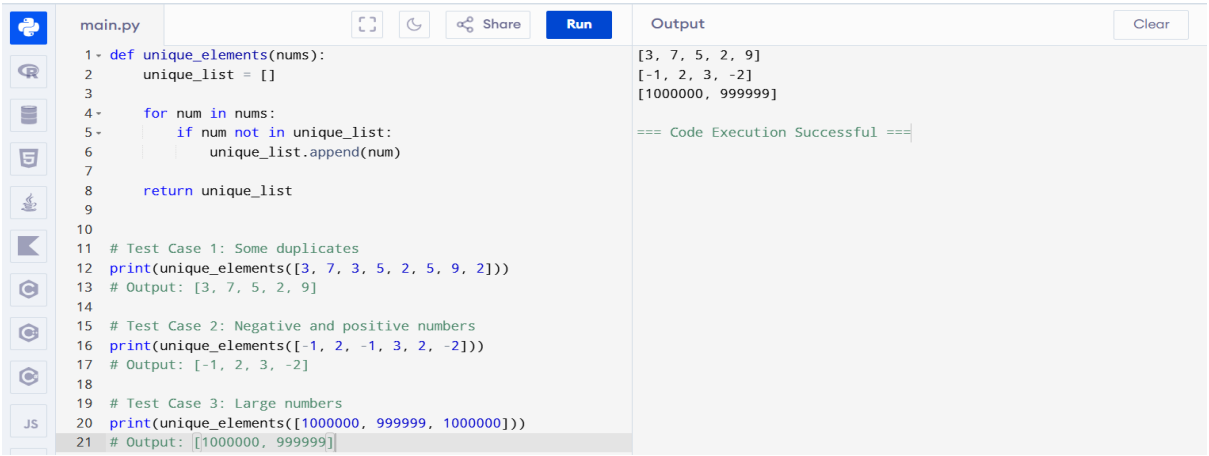
EXPERIMENT – 5



EXPERIMENT – 6



EXPERIMENT – 7



EXPERIMENT – 8

main.py

```python
def bubble_sort(nums):
    n = len(nums)

    for i in range(n):
        for j in range(0, n - i - 1):
            # swap if elements are in wrong order
            if nums[j] > nums[j + 1]:
                nums[j], nums[j + 1] = nums[j + 1], nums[j]

    return nums


# Example usage
arr = [5, 3, 8, 4, 2]
sorted_arr = bubble_sort(arr)
print("Sorted array:", sorted_arr)
```

Output

```
Sorted array: [2, 3, 4, 5, 8]

=== Code Execution Successful ===
```

EXPERIMENT – 9

main.py

```python
def binary_search(arr, key):
    # Step 1: Sort the array
    arr.sort()

    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == key:
            return mid   # position (0-based index)
        elif arr[mid] < key:
            left = mid + 1
        else:
            right = mid - 1

    return -1


# Test Case 1
arr1 = [3,4,6,-9,10,8,9,30]
key1 = 10
result1 = binary_search(arr1, key1)

```

Output

```
Element 10 is found at position 6
Element 100 is not found

=== Code Execution Successful ===
```

EXPERIMENT – 10

main.py

```python
def merge_sort(nums):
    if len(nums) <= 1:
        return nums

    mid = len(nums) // 2
    left = merge_sort(nums[:mid])
    right = merge_sort(nums[mid:])

    return merge(left, right)


def merge(left, right):
    sorted_list = []
    i = j = 0

    # Merge two sorted lists
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            sorted_list.append(left[i])
            i += 1
        else:
            sorted_list.append(right[j])
            j += 1

    # Add remaining elements
    sorted list.extend(left[i:])
```

Output

```
Sorted array: [1, 2, 3, 5]

=== Code Execution Successful ===
```

EXPERIMENT – 11

main.py

```python
def findPaths(m, n, N, i, j):
    MOD = 10**9 + 7

    # dp[x][y] = number of ways to reach cell (x, y)
    dp = [[0 for _ in range(n)] for _ in range(m)]
    dp[i][j] = 1

    count = 0

    for step in range(N):
        temp = [[0 for _ in range(n)] for _ in range(m)]

        for x in range(m):
            for y in range(n):
                if dp[x][y] > 0:
                    ways = dp[x][y]

                    # Move in 4 directions
                    for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
                        nx, ny = x + dx, y + dy

                        # If out of boundary → count it
                        if nx < 0 or nx >= m or ny < 0 or ny >= n:
                            count = (count + ways) % MOD
```

Output

```
6
12

=== Code Execution Successful ===
```

EXPERIMENT – 12

main.py

```python
def rob(nums):
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]

    # helper function (normal house robber)
    def rob_line(arr):
        prev = 0   # dp[i-2]
        curr = 0   # dp[i-1]

        for num in arr:
            temp = max(curr, prev + num)
            prev = curr
            curr = temp

        return curr

    # Case 1: exclude last house
    case1 = rob_line(nums[:-1])

    # Case 2: exclude first house
    case2 = rob_line(nums[1:])

    return max(case1, case2)
```

Output

```
3
4

=== Code Execution Successful ===
```

EXPERIMENT – 13

main.py

```python
def climb_stairs(n):
    if n <= 2:
        return n

    prev1 = 1  # ways to reach step 1
    prev2 = 2  # ways to reach step 2

    for i in range(3, n + 1):
        current = prev1 + prev2
        prev1 = prev2
        prev2 = current

    return prev2


# Test Cases
print(climb_stairs(4))  # Output: 5
print(climb_stairs(3))  # Output: 3
```

Output

```
5
3

=== Code Execution Successful ===
```

EXPERIMENT – 14

main.py

```python
def unique_paths(m, n):
    dp = [[1] * n for _ in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]


# Test Cases
print(unique_paths(7, 3))  # Output: 28
print(unique_paths(3, 2))  # Output: 3
```

Output

```
28
3

=== Code Execution Successful ===
```

EXPERIMENT – 15

main.py

```python
def largeGroupPositions(s):
    result = []
    n = len(s)
    i = 0

    while i < n:
        j = i

        # Move j while characters are the same
        while j < n and s[j] == s[i]:
            j += 1

        # Check if group length >= 3
        if j - i >= 3:
            result.append([i, j - 1])

        i = j  # Move to next group

    return result


# Test Cases
print(largeGroupPositions("abbxxxxzzy"))  # Output: [[3,6]]
print(largeGroupPositions("abc"))         # Output: []
```

Output

```
[[3, 6]]
[]

=== Code Execution Successful ===
```

EXPERIMENT – 16

main.py

```python
def gameOfLife(board):
    m, n = len(board), len(board[0])

    directions = [(-1,-1), (-1,0), (-1,1),
                  (0,-1),          (0,1),
                  (1,-1), (1,0), (1,1)]

    for i in range(m):
        for j in range(n):
            live_neighbors = 0

            for dx, dy in directions:
                ni, nj = i + dx, j + dy
                if 0 <= ni < m and 0 <= nj < n and abs
                    (board[ni][nj]) == 1:
                    live_neighbors += 1

            if board[i][j] == 1:
                if live_neighbors < 2 or live_neighbors > 3:
                    board[i][j] = -1
            else:
                if live_neighbors == 3:
                    board[i][j] = 2

    for i in range(m):
```

Output

```
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]

=== Code Execution Successful ===
```

EXPERIMENT – 17